FraudLens: detección de fraude en pagos digitales en tiempo real

Entregable 1: diagnóstico, requisitos y arquitectura

Dominio: fintech colombiana (billetera digital y pagos inmediatos)

Andrés Felipe Vélez Álvarez (1021923619; afveleza@eafit.edu.co)

Samuel Samper Cardona (1040180930; ssamperc@eafit.edu.co)

Sebastián Salazar Henao (1022003377; salazarh3@eafit.edu.co)

Hellen Yanes Doria (1026137704; hyanesd@eafit.edu.co)

Universidad EAFIT

ST1630 Sistemas Intensivos en Datos

Profesor: Andrés Sacre Alzate

11 de octubre de 2026

Repositorio: https://github.com/AndresVelez31/st1630-252 (rama `fraudlens-entregable1`, carpeta `proyecto-final/fraudlens/`)

FraudLens: detección de fraude en pagos digitales en tiempo real

1. Definición del problema

1.1 Contexto y problema de negocio

Los pagos digitales en Colombia crecen a un ritmo que los controles diarios ya no alcanzan. Bre-B, el sistema de pagos inmediatos del Banco de la República, procesó más de 1.700 millones de transacciones por más de $262 billones en su primer año y suma más de 36 millones de usuarios (Banco de la República, 2026); el 62 % de esas operaciones es menor a $50.000. Nequi, la billetera más grande del país, reporta más de 50 millones de transacciones diarias con 27 millones de usuarios (ABC Economía, 2025). El fraude crece con ellos: Asobancaria registró más de 218.000 reclamos por fraude en canales financieros solo en el primer semestre de 2025, y el 36,6 % de los colombianos dice haber sido víctima directa de un fraude en el último año (Suárez, 2026).

FraudLens es una billetera digital colombiana (empresa ficticia, proyecto académico) conectada a Bre-B que procesa pagos con QR, transferencias inmediatas y pagos a comercios. Hoy su equipo de riesgo detecta el fraude con reportes que se calculan al cierre del día sobre la base de datos operacional. Cuando un analista ve un patrón sospechoso, como una ráfaga de transferencias pequeñas hacia un comercio recién creado o un pago desde un dispositivo nunca visto en otra ciudad, el dinero ya salió: una transferencia inmediata se liquida en segundos y no se puede reversar.

El problema: los analistas de riesgo de FraudLens necesitan detectar patrones de fraude en menos de 2 minutos desde que ocurre la transacción sospechosa y, además, analizar el histórico completo para ajustar las reglas con evidencia, sin afectar el rendimiento de la app de pagos.

1.2 Usuarios del sistema y decisiones que toman

Tabla 1

Usuarios del sistema y decisiones que toman con los datos

Usuario	Decisión que toma con los datos	Urgencia
Analista de fraude	Bloquear una cuenta, pedir verificación adicional o descartar una alerta.	Minutos desde el evento
Jefe de riesgo	Ajustar umbrales de las reglas (montos, ventanas) según la tasa de fraude por comercio, ciudad y franja horaria.	Semanal
Equipo de comercios	Suspender o poner en revisión a comercios con montos sospechosos recurrentes.	Diaria
Auditoría y cumplimiento	Verificar de dónde salió una cifra y reconstruir el estado de los datos en una fecha pasada.	Bajo demanda

1.3 Preguntas que el sistema debe responder

Operacionales (tiempo real, alerta en menos de 2 minutos):

- P1. ¿Qué clientes superan 5 transacciones o $2.000.000 acumulados en una ventana de 5 minutos?
- P2. ¿Qué transacciones vienen de un dispositivo nunca visto para ese cliente y con un monto mayor a 3 veces su promedio de los últimos 30 días?
- P3. ¿Qué pagos involucran un comercio o un dispositivo de la lista de riesgo vigente?

Analíticas (capa Gold, datos con máximo 15 minutos de antigüedad):

- P4. ¿Cuál es la tasa de fraude (% de transacciones y % del monto) por comercio, ciudad y franja horaria, semana a semana?
- P5. ¿Cuál es la latencia promedio y en el percentil 95 entre la transacción y su alerta, y cuántas alertas dispara cada regla?
- P6. ¿Cuáles son los 10 comercios con mayor monto sospechoso en los últimos 7 días?

1.4 Por qué se necesita una arquitectura de datos distribuida

El volumen por sí solo no descarta una base relacional grande; lo que la descarta es la combinación de cinco condiciones (el dimensionamiento completo está en el Anexo A):

- **Volumen y velocidad.** Se diseña para 1.000 eventos/s en promedio, por encima del promedio de Nequi (≈580 transacciones/s) para absorber el crecimiento de Bre-B, con picos de 5.000 eventos/s en quincena, Día sin IVA y diciembre. Son unos 86 millones de eventos y 86 GB de JSON al día.
- **Estado por cliente en tiempo real.** P1 y P2 exigen ventanas de 5 minutos y promedios de 30 días por cliente, recalculados de forma continua. Hacerlo con consultas SQL sobre la tabla transaccional competiría con los pagos en curso.
- **Cargas mezcladas.** La app necesita escrituras rápidas y consistentes (OLTP) y los analistas, agregaciones sobre meses de histórico (OLAP). Separarlas evita que una consulta pesada degrade los pagos.
- **Variedad.** Llegan eventos JSON continuos, tablas relacionales de clientes y comercios y archivos CSV de listas de riesgo, cada uno con su formato, frecuencia y calidad.
- **Reprocesamiento y auditoría.** Cuando cambia una regla hay que recalcular sobre el histórico y poder reconstruir una fecha pasada, lo que exige almacenamiento inmutable y versionado, no tablas que se sobrescriben (Kleppmann, 2017).

2. Requisitos funcionales y no funcionales

2.1 Requisitos funcionales

Cada requisito se ubica en una capa del diagrama (Sección 3), es verificable en la demo y, cuando aplica, indica qué pregunta de la Sección 1.3 responde.

Tabla 2

Requisitos funcionales

ID	Requisito funcional	Capa	Verificación
RF-01	El sistema debe ingerir en tiempo real los eventos de transacción (pagos QR, transferencias inmediatas y pagos a comercios) en el topic `transacciones` de Kafka, con clave `id_cliente` y latencia menor a 5 s desde su generación.	Fuente → Ingestión	El productor sella cada evento; se mide el tiempo hasta que está en el topic.
RF-02	El sistema debe cargar cada día los catálogos de clientes y comercios desde PostgreSQL 16 hacia Bronze.	Ingestión batch	El DAG `catalogos_diario` registra la carga y Bronze tiene la versión del día.
RF-03	El sistema debe importar a Bronze cada versión nueva de los CSV de listas de riesgo (comercios y dispositivos) apenas se publique en `s3://.../landing/`.	Ingestión batch	Al subir un CSV, el sensor dispara la carga y aparece una versión nueva. Soporta P3.

Continúa en la página siguiente

ID	Requisito funcional	Capa	Verificación
RF-04	El sistema debe persistir los eventos crudos sin modificarlos en Bronze (Delta Lake en S3) con su topic, partición, offset y `ingest_ts`.	Bronze	Los `id_transaccion` distintos en Bronze coinciden con los eventos únicos producidos.
RF-05	El sistema debe validar, deduplicar por `id_transaccion` y estandarizar los eventos hacia Silver; el JSON ilegible va al topic `transacciones_dlq` y lo que incumple reglas de calidad, a una tabla de cuarentena con el motivo.	Procesamiento → Silver	Se inyectan duplicados, JSON roto y montos negativos: Silver solo tiene registros únicos y válidos y cada rechazo está en la DLQ o en cuarentena.
RF-06	El sistema debe contar transacciones y sumar montos por cliente en ventanas deslizantes de 5 min que avanzan cada minuto, y alertar al superar 5 transacciones o $2.000.000.	Procesamiento (streaming)	Una ráfaga de prueba dispara la alerta. Responde P1.
RF-07	El sistema debe alertar cuando una transacción venga de un dispositivo nunca visto para el cliente y su monto supere 3 veces su promedio de 30 días.	Procesamiento (streaming)	Caso con dispositivo nuevo y monto alto. Responde P2.
RF-08	El sistema debe alertar cuando un pago involucre un comercio o un dispositivo de la versión vigente de la lista de riesgo.	Procesamiento (join)	Pagos a un comercio listado disparan alerta. Responde P3.
RF-09	El sistema debe registrar cada alerta (regla, transacción, cliente, comercio, hora del evento y de la alerta) en MongoDB y en Silver sin duplicarla, y notificar por Amazon SNS solo las alertas nuevas.	Almacenamiento → Consumo	Tras reiniciar el job a mitad de un lote, el número de alertas y de correos no cambia.
RF-10	El sistema debe construir en Gold un esquema en estrella: hechos de transacciones y de alertas; dimensiones de cliente (SCD tipo 2), comercio, dispositivo, tiempo y regla.	Gold	Una consulta con joins entre hechos y dimensiones devuelve resultados correctos.
RF-11	El sistema debe calcular en Gold la tasa de fraude por comercio, ciudad y franja horaria, con corte semanal.	Batch → Gold	Consulta en Athena. Responde P4.
RF-12	El sistema debe calcular en Gold la latencia promedio y P95 entre transacción y alerta, y las alertas por regla.	Batch → Gold	Consulta en Athena. Responde P5.
RF-13	El sistema debe listar los 10 comercios con mayor monto sospechoso de los últimos 7 días.	Gold → Consumo	Consulta en Athena. Responde P6.
RF-14	El sistema debe permitir consultar Gold con SQL en Amazon Athena (tablas registradas en AWS Glue) y mostrar dashboards de fraude en Metabase, sin tocar la base operacional.	Consumo	Un analista ejecuta P4–P6 en Athena y las ve en Metabase.
RF-15	El sistema debe orquestar con Airflow tres DAG: `catalogos_diario` (diario o al llegar un CSV) `fraude_pipeline` (Silver → Gold, calidad y registro en Glue, cada 10 min) y `sesion_aws` (crea la infraestructura y restaura el respaldo al iniciar cada sesión de AWS). Un fallo debe detener los pasos dependientes y una tarea debe verificar que el streaming esté vivo.	Orquestación	Un fallo forzado detiene los pasos siguientes; si se detiene el streaming, Airflow lo reporta.
RF-16	El sistema debe reprocesar un rango de fechas con una versión nueva de las reglas (`rule_version`) escribiendo el resultado en Gold, sin modificar Bronze ni Silver.	Batch → Gold	Se reprocesa un día y se comparan las dos versiones.

Continúa en la página siguiente

ID	Requisito funcional	Capa	Verificación
RF-17	El sistema debe exponer las alertas en una API REST (FastAPI) consultable por cliente, regla y rango de tiempo.	Consumo	`GET /alertas` devuelve la alerta de la ráfaga de prueba. Soporta P1–P3.

2.2 Requisitos no funcionales

Los valores parten del dimensionamiento del Anexo A y de la meta de detección en menos de 2 minutos. La terminología de CAP y de garantías de entrega sigue a Kleppmann (2017); el tratamiento de datos personales, a la Ley 1581 de 2012.

Tabla 3

Requisitos no funcionales

ID / concepto	Requisito no funcional (valor concreto)	Cómo se verifica
RNF-01 Escalabilidad	Capacidad de diseño: 5.000 eventos/s sostenidos 30 min sin que el lag del consumidor crezca. El topic tiene 12 particiones (hasta 12 tareas de Spark en paralelo, ≈420 eventos/s por partición en el pico). Crecer a 10.000 eventos/s debe requerir solo más brokers y ejecutores, o mover el cómputo a EMR, sin cambiar código ni el almacenamiento en S3. Las particiones se fijan desde el inicio porque aumentarlas cambia la partición de cada `id_cliente` y rompe el orden por cliente.	Prueba de carga escalonada en el portátil de la demo; se reportan el máximo sostenido y el lag.
RNF-02 Disponibilidad (CAP)	La posición CAP se define por componente. **Kafka: CP**, con 3 brokers, replicación 3, `acks=all` y `min.insync.replicas=2`: con menos de 2 réplicas sincronizadas rechaza escrituras antes que aceptar pagos que puedan perderse. **S3: CP**, con consistencia de lectura tras escritura (Lab 1a). **MongoDB: CP en escritura y disponibilidad priorizada en lectura**: replica set de 3 nodos, escrituras con `w=majority` y lecturas de los analistas con `secondaryPreferred`, que siguen respondiendo si cae el primario (ADR-04). Metas: 99,5 % en ingestión y 99 % en consulta de alertas.	Se detiene un broker y luego el primario de MongoDB: la ingestión y la consulta siguen; con dos brokers detenidos, el productor recibe error en vez de perder datos.
RNF-03 Consistencia (garantía de entrega)	Ingestión at-least-once (`acks=all`, reintentos, productor idempotente). De Kafka a Silver y Gold el efecto es exactly-once: checkpoint de Spark junto a los datos en S3 y escrituras idempotentes con `MERGE` por `id_transaccion`. Las alertas se escriben con *upsert* sobre un `_id` determinista (hash de cliente, regla y ventana, o de la transacción), y SNS solo notifica cuando el *upsert* insertó un documento nuevo. Bronze admite duplicados; Silver, Gold, MongoDB y los correos, cero.	Se mata el job a mitad de un micro-batch: los conteos de Silver, MongoDB y correos no cambian.
RNF-04 Latencia	Evento → alerta: 30 s en P95 y 60 s como máximo (trigger de 5 s; R1 emite en modo `update` sin esperar el cierre de la ventana). Evento → Gold: 15 min (`fraude_pipeline` cada 10 min). Consultas de Athena sobre 30 días: menos de 10 s, gracias a la partición por fecha y a `OPTIMIZE ZORDER` (Lab 1b).	Hora del evento contra hora de la alerta (RF-12) y tiempo de ejecución de Athena.
RNF-05 Tolerancia a fallos	Si cae un broker, Kafka sigue sin perder escrituras confirmadas. Si cae el job de Spark, el contenedor se reinicia y retoma desde su checkpoint en menos de 5 min (RPO = 0 para eventos en Kafka, retenidos 7 días). Si termina la sesión de AWS, el sandbox borra el bucket: `sesion_aws` lo recrea y restaura el último respaldo en menos de 15 min, y lo que falte se reprocesa desde Kafka sin duplicar gracias al `MERGE`. Airflow reintenta cada tarea 3 veces con espera exponencial y luego detiene los dependientes y notifica.	Se apaga un broker, el primario de MongoDB y el job de streaming durante la demo y se cronometra la recuperación; se ensaya `sesion_aws` desde cero.

Continúa en la página siguiente

ID / concepto	Requisito no funcional (valor concreto)	Cómo se verifica
RNF-06 Trazabilidad	Silver y Gold conservan `source_system`, `kafka_partition`, `kafka_offset`, `ingest_ts`, `batch_id` y `rule_version`. Delta conserva 90 días de historial para time travel (en el sandbox, mientras el respaldo lo preserve). Toda cifra de Gold se rastrea hasta sus eventos de Bronze.	Se sigue una cifra de Gold hasta Bronze y se consulta `VERSION AS OF`.
RNF-07 Seguridad y privacidad	Documento, teléfono y correo se reemplazan desde Silver por un HMAC-SHA-256 con clave fuera del repositorio (un hash simple se revierte por fuerza bruta). S3 cifra en reposo (SSE-S3) y bloquea el acceso público; las API de AWS usan TLS. Las credenciales temporales del sandbox van en un `.env` excluido de Git. Solo auditoría lee Bronze con datos originales (Ley 1581 de 2012).	Las consultas de Athena y Metabase no devuelven datos personales en claro.
RNF-08 Calidad de datos	Los registros inválidos no detienen el pipeline: van a la DLQ o a cuarentena con su motivo; si la tasa de rechazo de una ejecución supera 1 %, `quality_checks` falla y notifica.	Se inyectan registros inválidos y se revisa la tasa.
RNF-09 Costo	El proyecto completo (desarrollo, pruebas y demo) debe costar menos de USD 50, el crédito de AWS Academy. Solo se usan servicios que cobran por uso (S3, Athena, Glue, SNS); no se crean EC2, volúmenes EBS, IP elásticas, RDS, snapshots, secretos ni métricas de CloudWatch, que cobran aunque no haya instancias encendidas (Anexo B).	Revisión del presupuesto del sandbox al iniciar cada sesión (AWS Budgets se actualiza cada 8 a 12 h).

3. Diagrama de arquitectura de datos por capas

La Figura 1 (página siguiente) muestra las cinco capas del pipeline como franjas horizontales, de la fuente al consumo, y usa la notación oficial de AWS (AWS Architecture Icons de draw.io) para el despliegue. A la izquierda, el grupo *equipo local* reúne lo que corre en contenedores con Docker Compose: productor, Kafka, Spark, Airflow, PostgreSQL, MongoDB, Metabase y FastAPI, cada uno con su perfil (`streaming`, `batch` o `consumo`). A la derecha, los grupos *AWS Cloud* y *Region* contienen los servicios administrados del sandbox: el bucket de S3 con los prefijos Bronze, Silver y Gold, Glue Data Catalog, Athena, SNS y la pila de CloudFormation que los crea en cada sesión. Cada componente nombra la herramienta concreta. Los archivos fuente están en el repositorio (`arquitectura.drawio`, `arquitectura.png` y `arquitectura.svg`) y el diagrama se puede editar en línea desde el enlace de la nota.

4. Justificación de decisiones técnicas (ADR)

Cada decisión no trivial se documenta con un ADR de cuatro partes. Los identificadores remiten a las Tablas 2 y 3 y a los riesgos de la Tabla 6. El ADR-06 (plataforma) condiciona a los demás, porque fija qué servicios permite el sandbox de AWS Academy.

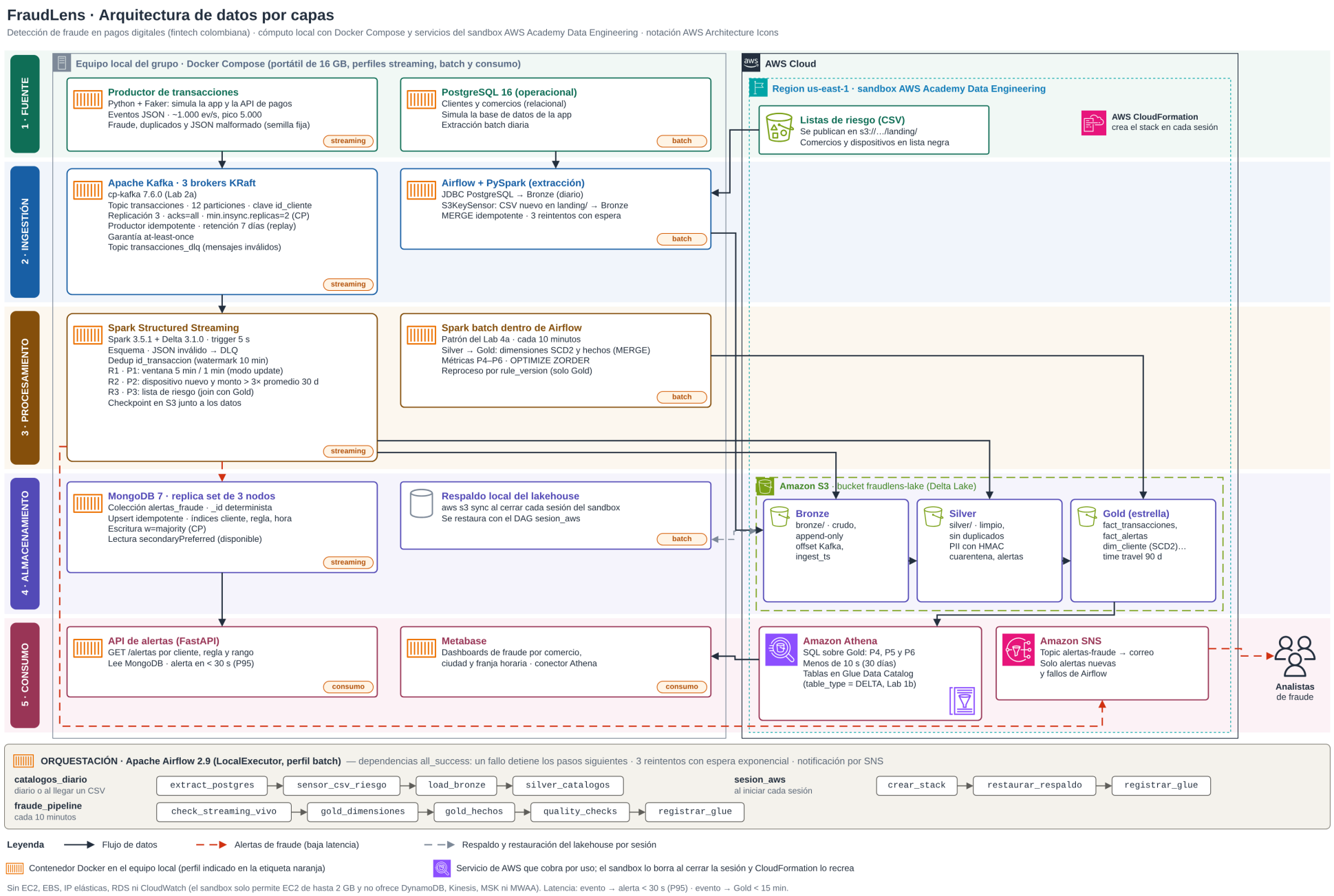
ADR-01: Apache Kafka con entrega at-least-once como capa de ingestión

Contexto. Llegan 1.000 eventos/s en promedio y 5.000 en el pico (RNF-01); un pago perdido no se recupera y, cuando cambia una regla, hay que releer el flujo sin pedirle los datos otra vez a la app (RF-16). Se necesita un registro durable, particionado y reproducible, desacoplado de la base operacional.

Alternativas consideradas. (a) *Escribir directo en PostgreSQL*: compite con los pagos (OLTP) y no sostiene 5.000 escrituras/s con ventanas calculadas encima. (b) *RabbitMQ*: borra el mensaje al consumirlo, así que no permite releer ni que varios consumidores lean a su ritmo. (c) *Amazon Kinesis* (Lab 2b): equivalente y administrado, pero no está habilitado en el sandbox del proyecto,

Figura 1

Arquitectura de datos por capas de FraudLens (cómputo local y servicios del sandbox de AWS Academy)



Nota. Enlace editable en draw.io: abrir el diagrama en app.diagrams.net. Archivo fuente: `arquitectura.drawio`.

cobra por shard-hora encendido y no corre en el `docker-compose.yml` que exige el E2 (ADR-06).

Decisión. Kafka en modo KRaft con imagen `cp-kafka:7.6.0` (la del Lab 2a), 3 brokers y un topic `transacciones` de 12 particiones con clave `id_cliente`: los eventos de un cliente caen en la misma partición y conservan su orden, que es lo que necesitan las reglas con estado por cliente. Replicación 3, `acks=all`, `min.insync.replicas=2`, productor idempotente y retención de 7 días. Frente a CAP la ingestión es **CP**: ante la pérdida de réplicas sincronizadas Kafka rechaza escrituras. La garantía configurada es **at-least-once**; el efecto exactly-once se logra aguas abajo con `MERGE` idempotente (RNF-03), el mismo patrón del Lab 2a. Se descartó at-most-once porque pierde pagos, y las transacciones de Kafka porque añaden complejidad que el `MERGE` ya resuelve.

Consecuencias. Se sacrifica disponibilidad de escritura si caen 2 de 3 brokers. Los duplicados llegan a Bronze y se eliminan en Silver. Las 12 particiones no se pueden aumentar sin romper el orden por cliente durante la transición. Los 3 brokers comparten el equipo local: toleran la caída de un proceso, no la de la máquina; en producción irían en hosts separados o en Amazon MSK.

ADR-02: Spark Structured Streaming frente a Flink o batch puro

Contexto. R1 a R3 necesitan estado por cliente en ventanas deslizantes de 5 min, cruces con listas de riesgo y alertas en 30 s (P95, RNF-04). El mismo equipo debe construir Silver y Gold en batch (RF-10 a RF-13) y los cuatro deben poder defender todo el código.

Alternativas consideradas. (a) *Apache Flink*: procesa evento por evento en milisegundos y maneja mejor el estado, pero obliga a operar un segundo motor además de Spark, y esa latencia no se necesita con una meta de 30 s. (b) *Batch puro cada 10 min*: no cumple los 2 minutos y la guía no acepta simular el streaming con batch. (c) *Kafka Streams*: reduce piezas, pero es solo JVM y no se integra con Delta ni con PySpark.

Decisión. Spark Structured Streaming 3.5.1 con Delta 3.1.0 (la combinación del Lab 4a) y trigger de 5 s. Se deduplica con `dropDuplicatesWithinWatermark` (watermark de 10 min) y se escribe con `foreachBatch` y `MERGE`. R1 usa ventanas de 5 min con avance de 1 min en modo `update`: emite en cada micro-batch sin esperar el cierre de la ventana, que en modo `append` añadiría minutos. R2 y R3 hacen join stream-static con las dimensiones de Gold. El checkpoint vive en S3 junto a los datos, así que ambos se respaldan y restauran juntos.

Consecuencias. Es micro-batch: la latencia mínima es de segundos. Un dispositivo visto después de la última actualización de Gold puede marcarse como nuevo durante hasta 10 min (riesgo aceptado en la Tabla 6). El estado de las ventanas consume memoria del equipo local, y los lotes de 5 s crean archivos pequeños que hay que compactar. Si el volumen se multiplicara por 100, el primer cambio sería evaluar Flink para R1.

ADR-03: Delta Lake sobre S3 con esquema en estrella en Gold

Contexto. Se necesita almacenamiento inmutable y versionado para reprocesar y auditar (RNF-06), escrituras idempotentes (RNF-03) y consultas de 30 días en menos de 10 s (RNF-04). P4 a P6 son agregaciones por comercio, ciudad y franja horaria.

Alternativas consideradas. (a) *Parquet sin formato de tabla*: sin ACID ni `MERGE`; deduplicar obliga a reescribir carpetas enteras. (b) *Apache Iceberg*: prestaciones similares, pero el equipo ya usó Delta con Glue y Athena en el Lab 1b, y cambiar suma aprendizaje sin resolver ningún requisito. (c) *Amazon Redshift* (permitido en el sandbox con ra3.large): cobra por hora encendido y mezcla almacenamiento con cómputo.

Decisión. Delta Lake en Amazon S3 con capas Bronze, Silver y Gold, consultado con Athena tras registrar las tablas en el Glue Data Catalog (`table_type = DELTA`; Amazon Web Services, s. f.). Delta aporta ACID, `MERGE` y 90 días de time travel. Gold es un esquema en estrella: `fact_transacciones` y `fact_alertas` con `dim_cliente` (SCD tipo 2), `dim_comercio`, `dim_dispositivo`, `dim_tiempo` y `dim_regla`. Es adecuado porque P4 a P6 son agregaciones sobre hechos filtradas por dimensiones y el modelo reduce joins frente a uno normalizado. Regla clave:

un único escritor por tabla. Delta OSS en S3 solo garantiza transacciones si las escrituras de una tabla salen de un mismo driver de Spark (Delta Lake, s. f.). Por eso el streaming es dueño de Bronze, Silver y las alertas, el batch es dueño de los catálogos y de Gold, y el reproceso (RF-16) escribe en Gold sin tocar Silver.

Consecuencias. Si en el futuro dos jobs tuvieran que escribir la misma tabla, haría falta el `S3DynamoDBLogStore`, que requiere DynamoDB, no disponible en el sandbox. Athena solo lee Delta: toda escritura pasa por Spark. Hay que compactar con `OPTIMIZE` y vigilar el esquema, porque un `mergeSchema` sin control rompe a los consumidores. El modelo en estrella exige diseñar las dimensiones por adelantado y es menos flexible para preguntas no previstas.

ADR-04: MongoDB como almacén de alertas y su posición CAP

Contexto. Las alertas se consultan por cliente, regla y rango de tiempo con baja latencia (RF-09, RF-17). El streaming las escribe varias veces por segundo en los picos, sus campos varían por regla y los analistas deben poder consultarlas aunque falle un nodo.

Alternativas consideradas. (a) *Amazon DynamoDB*: encajaba por cobrar por uso, pero no está habilitado en el sandbox del proyecto (AWS Academy, 2022). (b) *PostgreSQL*: añade carga a una base de tipo OLTP y su esquema rígido encaja mal con alertas que varían por regla. (c) *Cassandra*: AP y excelente para escrituras masivas, pero el volumen de alertas es bajo y su modelado por consultas es más costoso para un equipo de cuatro. (d) *Consultar Gold con Athena*: segundos de latencia y 10 min de frescura, insuficiente para la operación.

Decisión. MongoDB 7 en contenedores, como replica set de 3 nodos, con la colección `alertas_fraude`, un `_id` determinista (hash de `id_transaccion` o de cliente, regla y ventana) e índices por `id_cliente`, `regla` y `ts_alerta`. Cada alerta se escribe con *upsert*, lo que la hace idempotente y permite notificar por SNS solo cuando se insertó un documento nuevo. **Posición CAP:** MongoDB es CP por defecto: hay un único primario y, ante una partición, el lado sin mayoría no acepta escrituras. Las escrituras usan `w=majority` para no perder alertas confirmadas, y las lecturas de los analistas usan `readPreference=secondaryPreferred`: si el primario cae, los secundarios siguen respondiendo con consistencia eventual. Es decir, CP en escritura y disponibilidad priorizada en lectura, no que MongoDB sea AP: una alerta que aparece segundos después no cambia la decisión del analista, mientras que perderla sí.

Consecuencias. Durante una elección de primario (segundos) no se pueden escribir alertas; el checkpoint de Spark y el `_id` determinista permiten reintentar sin duplicados. Una lectura desde un secundario puede no mostrar la alerta más reciente. Los 3 nodos comparten el equipo local, así que la tolerancia es a la caída de un proceso, no de la máquina; consumen memoria, que se limita con una caché reducida. Se pierden joins y transacciones complejas, lo que no afecta a este caso.

ADR-05: Apache Airflow y modelado de dependencias de los DAG

Contexto. Los pasos batch deben dispararse según dependencias, fallar de forma limpia si un paso anterior falla (RF-15) y dejar evidencia de su estado. Hay dos ritmos: catálogos (diario o al llegar un CSV) y Gold (cada 10 min, para cumplir los 15 min de RNF-04).

Alternativas consideradas. (a) *Cron y scripts*: sin dependencias, reintentos con espera ni interfaz para diagnosticar. (b) *Dagster o Prefect*: válidas, pero Airflow es la referencia del curso (Lab 4a). (c) *Amazon MWAA o Step Functions*: MWAA no está en el sandbox y cobra por hora; Step Functions sí está, pero no orquesta procesos que corren en el equipo local.

Decisión. Airflow 2.9 con LocalExecutor y la imagen del Lab 4a (Java 17, PySpark 3.5.1 y delta-spark 3.1.0), de modo que los jobs batch corren dentro del worker de Airflow. Tres DAG:

- `catalogos_diario` (diario o al llegar un CSV): `extract_postgres` → `sensor_csv_riesgo` (sensor sobre S3) → `load_bronze` → `silver_catalogos`.
- `fraude_pipeline` (cada 10 min): `check_streaming_vivo` → `gold_dimensiones` → `gold_hechos` → `quality_checks` → `registrar_glue`.
- `sesion_aws` (al iniciar cada sesión del sandbox): `crear_stack` (CloudFormation) → `restaurar_respaldo` → `registrar_glue`; su tarea `respaldar` copia el lakehouse al equipo antes de cerrar la sesión.

Las dependencias usan `all_success`, así que un fallo detiene los pasos siguientes en vez de propagar datos corruptos; cada tarea reintenta 3 veces con espera exponencial y luego notifica por SNS. El streaming no es una tarea del DAG porque corre continuo; `check_streaming_vivo` lee el latido que el job publica y falla si lleva más de 2 min sin avanzar. Todas las tareas son idempotentes (`MERGE`), así que un `backfill` no duplica datos.

Consecuencias. Airflow es un planificador por lotes: Gold queda hasta 15 min atrás. LocalExecutor no escala horizontalmente; en producción se usaría CeleryExecutor o MWAA. El pipeline lineal no paraleliza las dimensiones, algo optimizable si una ejecución se acerca a los 10 min.

ADR-06: Plataforma: cómputo local y servicios del sandbox de AWS

Contexto. El pipeline necesita correr a la vez un broker, Spark, el lakehouse, un orquestador y el componente libre, con un `docker-compose.yml` que lo levante desde cero y una demo en vivo. El equipo usa el sandbox de AWS Academy Data Engineering, con USD 50 de presupuesto y tres restricciones decisivas (AWS Academy, 2022): solo permite EC2 nano, micro y small de las series t2 y t3 (máximo 2 GB de RAM), y EMR con los mismos tamaños; no incluye DynamoDB, Kinesis, MSK ni MWAA; y **al terminar cada sesión borra todos los datos y recursos creados**.

Alternativas consideradas. (a) *Todo en AWS* (EC2 o EMR para Kafka, Spark y Airflow): con instancias de 2 GB no caben ni un broker con Spark, y los servicios administrados equivalentes no están disponibles. (b) *Todo local* (MinIO en vez de S3): no cuesta y no depende de la red, pero se pierde el lakehouse sobre almacenamiento de objetos real y la consulta con Glue y Athena de los Labs 1a y 1b. (c) *Híbrido*: cómputo local y almacenamiento y consulta en los servicios del sandbox que cobran por uso.

Decisión. Híbrido. Kafka, Spark, Airflow, PostgreSQL, MongoDB, Metabase y FastAPI corren con un mismo `docker-compose.yml` con perfiles `streaming`, `batch` y `consumo` en el equipo de la demo (16 GB de RAM). En AWS solo se usan servicios que cobran por uso: S3 para Bronze, Silver y Gold; Glue Data Catalog y Athena para consultar Gold; y SNS para notificar. Como el sandbox borra todo al cerrar la sesión, la infraestructura de AWS es código: una plantilla de CloudFormation crea el bucket, la base de datos de Glue, el *workgroup* de Athena y el topic de SNS, y el DAG `sesion_aws` la levanta, restaura el último respaldo del lakehouse y registra las tablas en Glue. Antes de cerrar la sesión, `respaldar` copia el bucket al equipo con `aws s3 sync`. Las credenciales temporales de la sesión van en un `.env` excluido de Git.

Consecuencias. El equipo local es el punto único de falla del cómputo y debe tener al menos 16 GB de RAM. Cada sesión empieza con unos minutos de `sesion_aws`; si se olvida el respaldo, lo perdido se reprocesa desde Kafka (7 días de retención) y el `MERGE` evita duplicados. La demo depende de la red del salón; el plan B es un perfil `offline` con MinIO, el mismo código con otro *endpoint*. El costo baja a unos USD 5, porque no hay instancias ni discos que cobren por hora (Anexo B). En producción el mismo diseño se movería a EMR o MSK sin cambiar el almacenamiento.

5. Plan de implementación

El sistema se implementa entre S12 y S15. Cada integrante es dueño de un bloque de extremo a extremo (código, pruebas y el ADR que lo sustenta) y es par revisor de otro, para que los cuatro puedan defender cualquier parte en S16.

5.1 Distribución de responsabilidades

Tabla 4

Distribución de responsabilidades entre los integrantes

Integrante	Componentes que implementa	Requisitos	Par revisor
Andrés Felipe Vélez Álvarez	Infraestructura e ingestión. `docker-compose.yml` con perfiles; plantilla de CloudFormation y DAG `sesion_aws` (crear, restaurar y respaldar); Kafka KRaft (3 brokers, topics `transacciones` y `transacciones_dlq`); productor Python/Faker con escenarios de fraude, duplicados y registros malformados (semilla fija); prueba de carga; control de costos.	RF-01, RNF-01, RNF-02 (Kafka), RNF-05, RNF-09	Samuel Samper Cardona
Samuel Samper Cardona	Procesamiento streaming. Kafka → Bronze → Silver (dedup con watermark, DLQ, cuarentena, `MERGE`); reglas R1–R3; escritura idempotente de alertas en MongoDB y Silver; notificación SNS; checkpoints en S3; latido del job; medición del P95.	RF-04 a RF-09, RNF-03, RNF-04, RNF-08	Sebastián Salazar Henao
Sebastián Salazar Henao	Batch y lakehouse. Extracción JDBC de PostgreSQL y carga de CSV a Bronze; Silver de catálogos; Gold (dimensiones SCD2 y hechos); consultas P4–P6; registro en Glue; `OPTIMIZE ZORDER`; reproceso por `rule_version`; HMAC de datos personales; time travel.	RF-02, RF-03, RF-10 a RF-13, RF-16, RNF-06, RNF-07	Hellen Yanes Doria
Hellen Yanes Doria	Orquestación y consumo. DAG `catalogos_diario` y `fraude_pipeline` (dependencias, reintentos, fallo limpio, `check_streaming_vivo`, `quality_checks`); replica set de MongoDB e índices; API FastAPI y Metabase; consultas en Athena; README.	RF-14, RF-15, RF-17, RNF-02 (MongoDB)	Andrés Felipe Vélez Álvarez

Acuerdos de trabajo: el contrato de datos (esquema JSON del evento y nombres de columnas) se congela en S12 y solo cambia por un PR aprobado por los dueños afectados. Todo cambio entra por Pull Request revisado por el par. Cada semana hay 30 minutos en los que cada dueño explica su bloque a los demás, y cada integrante registra en `/docs/bitacora_ia.md` lo que delega a agentes de IA.

5.2 Cronograma e hitos

Tabla 5

Cronograma e hitos de implementación

Semana	Trabajo planeado	Hito verificable
S12	Entrega del E1. Contrato de datos; repositorio por componentes; matriz de versiones; `docker-compose` mínimo (Kafka, Airflow, PostgreSQL, MongoDB); plantilla de CloudFormation; medición de memoria en el equipo de la demo.	H1: E1 entregado; `docker compose up` funciona en los 4 equipos y `sesion_aws` crea el bucket en el sandbox.

Continúa en la página siguiente

Semana	Trabajo planeado	Hito verificable
S13	Ajustes por la retroalimentación del E1. Streaming Kafka → Bronze → Silver con dedup, DLQ y cuarentena; catálogos y CSV a Bronze y Silver; registro de una tabla Delta en Glue y consulta en Athena; esqueleto de los DAG.	H2: Silver tiene tantos `id_transaccion` únicos como eventos producidos; Athena consulta una tabla Delta.
S14	Reglas R1–R3 con MongoDB y SNS; dimensiones y hechos de Gold; DAG completos con reintentos; API de alertas. Asesoría opcional con un problema documentado.	H3: primera alerta de extremo a extremo (evento → MongoDB → correo) y P4 respondida en Athena.
S15	Dashboards en Metabase; reproceso por `rule_version`; pruebas de fallo (broker, streaming, DAG); prueba de carga y mediciones para las slides. Congelamiento a mitad de semana; README, bitácora, slides y simulacro de defensa cruzada.	H4: el pipeline corre desde cero siguiendo solo el README; P1–P6 respondidas; PR final listo.
Día antes de S16	Ensayo de 25 min con la sesión del sandbox iniciada y `sesion_aws` ejecutado; PR final abierto antes de las 11:59 p. m.	H5: la demo corre en vivo sin pasos manuales.

5.3 Riesgos técnicos y plan de mitigación

Tabla 6

Riesgos técnicos y plan de mitigación

Riesgo	Impacto	Mitigación	Responsable
El sandbox borra el bucket y las tablas de Glue al terminar la sesión, incluso durante la demo.	Alto	Infraestructura como código (CloudFormation) y DAG `sesion_aws` ensayado desde cero; respaldo al cerrar cada sesión; iniciar la sesión 1 h antes de la demo y renovarla con *Start Lab*; plan B offline con MinIO.	Andrés Felipe Vélez Álvarez
Se agota el presupuesto de USD 50.	Medio	Solo servicios que cobran por uso; ninguno de los que cobran sin instancias encendidas (EBS, IP elásticas, RDS, snapshots, CloudWatch); revisión del presupuesto al iniciar cada sesión.	Andrés Felipe Vélez Álvarez
El stack local (3 brokers, Spark, Airflow, MongoDB $\times 3$, Metabase) no cabe en la RAM del portátil.	Alto	Medir en S12; límites de memoria por contenedor, heap de 512 MB por broker, caché reducida en MongoDB, perfiles de Compose; la demo corre en el equipo con más RAM.	Andrés Felipe Vélez Álvarez
Versiones incompatibles entre Spark, Delta, `hadoop-aws` y el conector de Kafka.	Alto	Matriz fija desde S12 (Spark 3.5.1, Delta 3.1.0, `hadoop-aws` 3.3.4, `cp-kafka` 7.6.0), imágenes propias con los JAR y ninguna etiqueta `latest`.	Andrés Felipe Vélez Álvarez
La alerta de R1 supera 30 s si la ventana emite en modo `append`.	Alto	Modo `update` con `foreachBatch`, trigger de 5 s; medir el P95 desde S14.	Samuel Samper Cardona
Al reiniciar el job se duplican alertas o correos.	Medio	`_id` determinista con *upsert*; `MERGE` en Delta; prueba de reinicio forzado en S15.	Samuel Samper Cardona
Athena no lee una tabla Delta o el registro en Glue falla.	Medio	Prueba de concepto en S13 (patrón del Lab 1b); plan B: responder P4–P6 con Spark SQL desde Airflow.	Sebastián Salazar Henao
Las credenciales temporales caducan durante una carga o la demo.	Medio	Renovar con *Start Lab* antes de que el contador llegue a cero; las tareas son idempotentes y se reintentan.	Hellen Yanes Doria
Un integrante no puede defender partes que no implementó (40 % del E2).	Alto	Par revisor por bloque, PR revisados por otro integrante, explicación semanal y simulacro de preguntas en S15.	Todos

Referencias

ABC Economía. (2025, 24 de noviembre). *Nequi se prepara para pico de 80 millones de transacciones diarias*. https://abceconomia.co/2025/11/24/nequi-se-prepara-para-pico-de-80-millones-de-transacciones-diarias/

Amazon Web Services. (s. f.). *Get started with Delta Lake tables*. Amazon Athena User Guide. https://docs.aws.amazon.com/athena/latest/ug/delta-lake-tables-getting-started.html

AWS Academy. (2022). *AWS Academy Data Engineering: Sandbox* [Instrucciones del laboratorio]. Amazon Web Services.

Banco de la República. (2026, 6 de octubre). *Bre-B cumple un año transformando la forma de realizar pagos y transferencias en Colombia*. https://banrep.gov.co/es/noticias/bre-b-cumple-un-anio-en-colombia

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587.

Delta Lake. (s. f.). *Storage configuration: Amazon S3*. https://docs.delta.io/latest/delta-storage.html

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Sacre Alzate, A. (2026). *ST1630 Sistemas Intensivos en Datos: laboratorios 1a, 1b, 2a, 2b y 4a* [Repositorio de GitHub]. Universidad EAFIT. https://github.com/asacrea/st1630-252

Suárez, M. C. (2026, 3 de enero). Fraude financiero aumenta en Colombia: el panorama y cómo prevenirlo en 2026. *Bloomberg Línea*. https://www.bloomberglinea.com/latinoamerica/colombia/fraude-financiero-aumenta-en-colombia-el-panorama-y-como-prevenirlo-en-2026/

Villamil, M. (2026, 6 de mayo). Bre-B disparó sus transacciones en Colombia y ya mueve hasta 5 millones de operaciones diarias. *Infobae*. https://www.infobae.com/colombia/2026/05/06/bre-b-disparo-sus-transacciones-en-colombia-y-ya-mueve-hasta-5-millones-de-operaciones-diarias/

Anexo A. Dimensionamiento

Magnitud	**Valor**	**Cálculo o supuesto**
Referencia de mercado	≈580 tx/s	Nequi: más de 50 millones de transacciones diarias / 86.400 s (ABC Economía, 2025).
Promedio de diseño	1.000 eventos/s	Referencia con margen para el crecimiento de Bre-B, que triplicó su volumen diario en siete meses (Villamil, 2026).
Pico de diseño	5.000 eventos/s	5 veces el promedio: quincena, Día sin IVA, Día de las Velitas.
Eventos al día	≈86 millones	1.000×86.400 s.
Tamaño por evento	≈1 KB	JSON con 25 campos (cliente, comercio, dispositivo, geolocalización).
Bronze crudo	≈86 GB/día; ≈31 TB/año	Antes de compresión; en Parquet comprimido se estima 5 a 8 veces menos.
Carga por partición en el pico	≈420 eventos/s	5.000 / 12 particiones; un consumidor de Spark sostiene varios miles de eventos/s.
Retención en Kafka	7 días	Permite reprocesar si una sesión del sandbox termina sin respaldo.
Volumen de la demo	50 a 500 eventos/s	Lo que permite el portátil de la demo; se reporta el máximo medido.

Anexo B. Estimación de costos en el sandbox de AWS Academy (us-east-1)

Servicio	Supuesto de uso (S12 a S16, unas 15 sesiones)	Costo estimado
Amazon S3	Menos de 20 GB por sesión (se borra al cerrarla) y solicitudes PUT de los micro-batches (USD 0,005 por 1.000)	USD 2,00
Transferencia de salida	Respaldo con `aws s3 sync` al cerrar cada sesión, unos 2 GB $\times$ USD 0,09/GB	USD 2,70
Athena	USD 5 por TB escaneado; cientos de consultas de pocos MB	menos de USD 0,50
Glue Data Catalog	Pocas decenas de tablas, dentro de la capa gratuita	USD 0
Amazon SNS	Pocos cientos de correos	menos de USD 0,10
CloudFormation	Sin costo propio; solo se pagan los recursos que crea	USD 0
Total		$\approx$ **USD 5 de 50**

Nota. Precios de lista en us-east-1 a octubre de 2026. No se usan EC2, EBS, IP elásticas, RDS, snapshots, Secrets Manager ni métricas o registros de CloudWatch, que generan cargos aunque no haya instancias encendidas. El presupuesto visible en el laboratorio proviene de AWS Budgets y se actualiza cada 8 a 12 horas.

Anexo C. Bitácora de delegación

Tarea	¿Delegado?	Justificación
Definición del problema y preguntas P1–P6 (Sección 1)	Parcial	Redactada por el equipo. El agente (Claude) buscó cifras de mercado con su fuente (Banco de la República, Asobancaria, Nequi) y las incorporó.
Requisitos funcionales y no funcionales (Sección 2)	Parcial	Redactados por el equipo. El agente señaló inconsistencias (RF-04 frente a RNF-03, CAP por componente, RF faltante para la API) y los adaptó al despliegue en AWS.
Elección de tecnologías (Kafka, Spark, Delta Lake, Airflow)	No	Decisión de diseño central tomada por el equipo antes de usar el agente.
Adaptación al sandbox de AWS Academy (cómputo local; S3, Glue, Athena y SNS con CloudFormation)	Parcial	Propuesta por el agente tras revisar los laboratorios y las restricciones del sandbox; el equipo la aprobó. MongoDB se mantiene como lo eligió el equipo.
Redacción de los ADR (Sección 4)	Parcial	El equipo definió contexto, alternativas y decisión; el agente ayudó a redactar y a citar los conceptos del curso.
Plan de implementación (Sección 5)	Parcial	Reparto de bloques acordado por el equipo; el agente redactó el cronograma y la tabla de riesgos.
Dibujo del diagrama en draw.io (notación AWS) y exportación	Sí	El agente generó el .drawio con los íconos oficiales de AWS a partir del diseño del equipo (`build_diagram.py`). Bajo valor pedagógico.
Maquetación en LaTeX, referencias APA y anexos	Sí	Tarea de formato; bajo valor pedagógico.